

Cosmos Policy 学习笔记

-作者: Yang kunpeng

目录

第一章	Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning	1
1	起点：把视频 diffusion 看成统一生成接口	2
2	Latent Frame Injection: 把机器人变量变成“帧”	2
2.1	统一序列 $(s, a, s', V(s'))$	2
2.2	低维向量如何填满高维 latent volume	3
2.3	为什么需要 blank placeholder 和重复图像	4
3	同一个模型如何同时学 Policy、World Model 和 Value	6
3.1	为什么 policy 不只预测 a	6
3.2	为什么初始 batch 是 50/25/25	6
4	Value function: 监督信号从哪里来	7
5	为什么 demonstrations 不足以支持 planning	7
6	Model-Based Planning: Best-of- N 怎么运行	8
6.1	$V(s')$ 与 $Q(s, a)$ 的区别	9
7	EDM 噪声分布为什么必须为机器人控制重新设计	9
8	并行解码与自回归解码	10
9	几个值得记住的实验结论	11
10	把整篇方法串起来	11
11	值得长期保留的理解	12

Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning

Cosmos Policy 最值得记住的不是“视频模型加了一个动作 head”，而是它把机器人动作、proprioception、future observation 和 value 全部改写成视频 diffusion sequence 中的 latent frame。这样，同一个预训练视频模型可以通过不同 conditioning mask，在不增加三套独立网络的情况下分别扮演 policy、world model 和 value function。

这一章沿着“视频 diffusion 的原始接口为什么能够容纳机器人控制”这条主线展开：先复习 Cosmos-Predict2 的 EDM 目标，再看 latent frame injection 如何统一模态，随后解释联合训练、value 学习、on-policy rollout refinement、best-of- N planning，以及为什么机器人控制需要重新设计 EDM 的噪声分布。

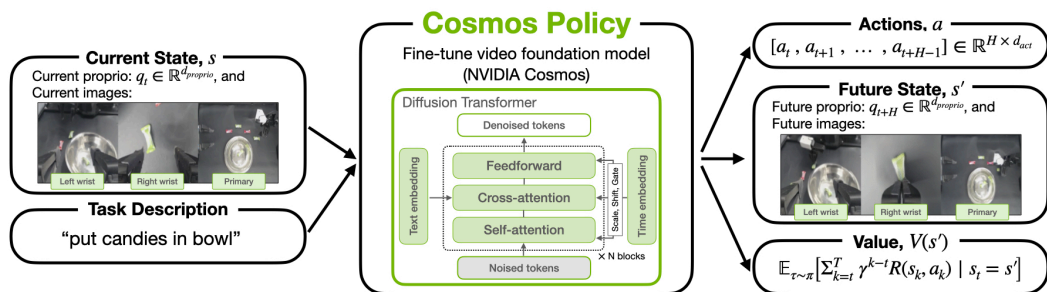


图 1.1: Cosmos Policy 的总体框架。一个预训练视频模型通过 latent injection 同时生成动作、未来状态和 value，并可进一步用 world model 与 value function 做 model-based planning。

1 起点：把视频 diffusion 看成统一生成接口

Cosmos Policy 初始化自 Cosmos-Predict2-2B-Video2World。原模型接收起始图像和文本描述，通过 Wan2.1 时空 VAE 把视频编码到连续 latent space，再由 Diffusion Transformer 在不同噪声水平上恢复 clean latent sequence。

若干干净 latent sequence 为 x_0 ，文本条件为 c ，噪声满足

$$n \sim \mathcal{N}(0, \sigma^2 I),$$

则 denoiser D_θ 的核心目标为

$$\mathcal{L}(D_\theta, \sigma) = \mathbb{E}_{x_0, c, n} \left[\|D_\theta(x_0 + n; \sigma, c) - x_0\|_2^2 \right]. \quad (1.1)$$

这说明预训练模型本质上已经掌握一种非常通用的操作：**给定一组 clean condition frames 与一组 noisy target frames，把目标恢复成 clean latent。**

Cosmos Policy 的核心问题于是变成：如果 action、proprioception、value 也能被表示成与视频 latent frame 形状相同的张量，那么能否直接复用这套 denoising machinery？答案就是 latent frame injection。

Wan2.1 tokenizer 把

$$(1 + T) \times H \times W \times 3$$

的视频压缩为

$$(1 + T') \times H' \times W' \times 16, \quad T' = \frac{T}{4}, \quad H' = \frac{H}{8}, \quad W' = \frac{W}{8}.$$

第一个 frame 不做时间压缩，因此后续的 placeholder 布局也需要尊重这一 tokenizer 结构。

2 Latent Frame Injection：把机器人变量变成“帧”

2.1 统一序列 $(s, a, s', V(s'))$

以包含 wrist camera 与两个 third-person cameras 的机器人为例，完整 latent sequence 可以包含 11 个 frame：blank placeholder、当前 proprioception、三路当前

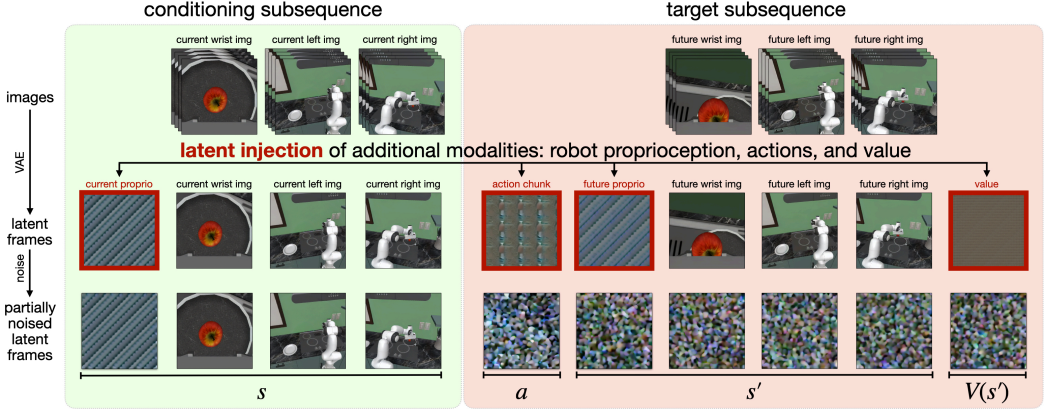


图 1.2: Latent frame injection 的原始示意图。动作、机器人状态、未来状态和 value 被写进与视频 latent 相同形状的位置；哪些 frame 保持 clean、哪些 frame 被加噪，决定当前训练的是哪一种功能。

图像、action chunk、future proprioception、三路 future images，以及 future state value。

从概率建模角度，它们按顺序组成

$$(s, a, s', V(s')).$$

其中 s 和 s' 是对机器人 observation 的简写，实际包含 proprioception 与多相机图像，而不是真正完全可观测的物理状态。

2.2 低维向量如何填满高维 latent volume

设 action chunk 的形状为

$$K \times d_{\text{act}},$$

单个 latent frame 的形状为

$$H' \times W' \times C'.$$

首先把每个 action dimension 归一化到 $[-1, +1]$ ，再展平成长度 Kd_{act} 的向量。随后重复

$$N_{\text{copy}} = \frac{H'W'C'}{Kd_{\text{act}}} \quad (1.2)$$

次，reshape 成 $H' \times W' \times C'$ ，并直接覆盖对应 placeholder latent。

proprioception 和 value 使用完全相同的思想，只是原始维度不同。推理时再反向操作：对重复 copies 求平均、反归一化，就能恢复 action；value 是标量，因此直接对整个 value latent volume 求平均即可。

Latent frame injection 的真正价值不是“编码技巧很巧”，而是它完全不改变视频 diffusion backbone 的输入输出类型。新模态只要能被包装成 latent frame，就可以继承预训练视频模型的全部生成结构。

2.3 为什么需要 blank placeholder 和重复图像

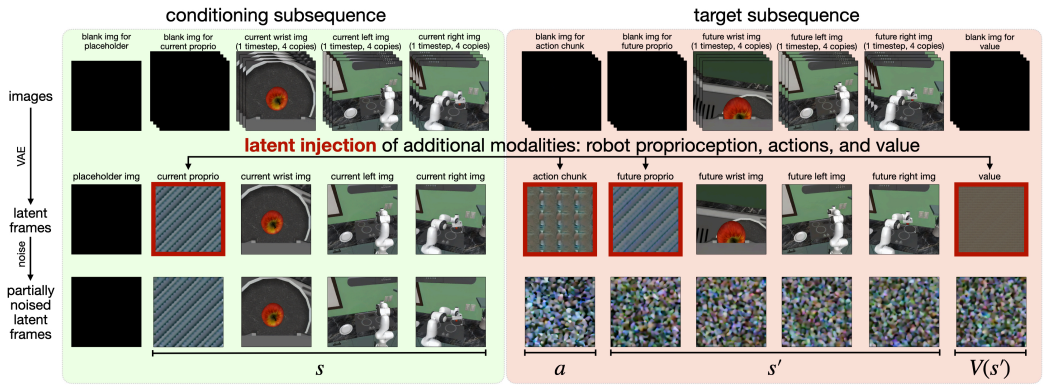


图 1.3: Latent injection 的详细原始图。blank images 先经过 VAE 形成 placeholder latent，再由 action、proprioception 和 value 覆盖；图像按 tokenizer 的时间压缩结构成组排列。

Wan2.1 的第一张图单独编码，后续图像按四帧一组做时间压缩。为了让当前状态与未来状态在 latent sequence 中拥有对称的结构，论文在序列起始位置保留一个 blank placeholder，并对每个 camera observation 构造四份相同图像，使每组四图在 VAE 后对应一个 latent frame。

因此这里的序列布局不是任意拼接，而是与预训练 tokenizer 的时空结构严格配合。



图 1.4: 联合训练的 conditioning scheme。完整 latent sequence 固定不变, clean/noisy mask 的不同组合决定当前样本训练 policy、world model 还是 value function。

3 同一个模型如何同时学 Policy、World Model 和 Value

对每个训练样本，统一写成

$$(s, a, s', V(s')).$$

Cosmos Policy 不需要三个 head，而是通过“哪些 frame 是 clean condition、哪些 frame 是 noisy target”来切换三个条件生成问题：

$$\text{Policy: } p(a, s', V(s') | s), \quad (1.3)$$

$$\text{World model: } p(s', V(s') | s, a), \quad (1.4)$$

$$\text{Value function: } p(V(s') | s, a, s'). \quad (1.5)$$

3.1 为什么 policy 不只预测 a

最简单的 imitation policy 只需要学 $p(a | s)$ 。Cosmos Policy 故意让 policy 同时预测

$$(a, s', V(s')).$$

这意味着动作生成必须与“这个动作以后会看到什么”和“这个未来大概有多好”一起被建模。future state 与 value 因而不仅是 planning 接口，也是 policy 训练时的 auxiliary supervision。

world model 同理：它不是只预测 $p(s' | s, a)$ ，而是联合预测 $p(s', V(s') | s, a)$ 。

3.2 为什么初始 batch 是 50/25/25

初始训练每个 batch 中：50% 样本来自 demonstrations，用于 policy；另外 50% 来自 rollout dataset，并平均分给 world model 与 value function。因此总体比例为

$$50\% : 25\% : 25\%.$$

初始阶段 rollout dataset 可以只是 demonstrations 的超集，并包含回放时失败的 demonstrations；后续 planning 阶段才会真正收集当前 policy 的 on-policy rollouts。

4 Value function: 监督信号从哪里来

机器人任务被写成有限时域 MDP

$$\langle \mathcal{S}, \mathcal{A}, T, R, H \rangle.$$

对策略 π , 状态 value 为

$$V^\pi(s_t) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=t}^H \gamma^{k-t} R(s_k, a_k) \mid s_t = s \right]. \quad (1.6)$$

论文采用 sparse reward: 在 $t < H$ 时 reward 为 0, 只在终点给任务得分。于是式 (1.6) 化为

$$V^\pi(s_t) = \mathbb{E}_{\tau \sim \pi} [\gamma^{H-t} R(s_H, a_H) \mid s_t = s]. \quad (1.7)$$

训练时直接使用观测到的 Monte Carlo return

$$G_t = \gamma^{H-t} R(s_H, a_H) \quad (1.8)$$

作为 value target。

因此不需要 TD bootstrapping: 一条 rollout 完成后, 轨迹中间每个位置都可以从最终 outcome 获得 label。

5 为什么 demonstrations 不足以支持 planning

imitation dataset 大多集中在成功轨迹附近。这样的数据适合训练“如何模仿专家”, 却不适合训练“这个偏离专家分布的候选动作会不会失败”。而 planning 恰恰需要 world model 和 value function 能比较多个候选动作, 其中不少动作会进入训练 demonstrations 没覆盖过的状态。

因此论文部署当前 Cosmos Policy 收集真实 rollout, 记录完整轨迹以及 success/failure 或 fractional score。随后从 base checkpoint 继续 fine-tune, 但把训练资源明显偏向 world model 与 value function:

$$90\% \text{ batch} \rightarrow \text{world model/value}, \quad 10\% \text{ batch} \rightarrow \text{policy}.$$

90% 再平均分成 world model 与 value function 两部分。

这一步之后采用 dual deployment:

- 原始 checkpoint 作为 **policy model**, 负责提出候选动作;
- rollout-refined checkpoint 作为 **planning model**, 负责未来状态预测和 value estimation。

这样避免了一个很实际的问题: 为了让 world/value 更懂失败状态而大量加入 on-policy 数据, 可能反过来破坏已经很强的 imitation policy。

6 Model-Based Planning: Best-of- N 怎么运行

给定当前状态 s , policy model 先生成 N 个 action proposals:

$$a^{(1)}, \dots, a^{(N)} \sim \pi(\cdot | s).$$

对每个候选动作, planning model 预测 future state

$$\hat{s}'^{(i)} \sim p(s' | s, a^{(i)}),$$

再估计

$$\hat{V}^{(i)} \approx V(\hat{s}'^{(i)}).$$

最终选择

$$a^* = \arg \max_{a^{(i)}, i=1, \dots, N} \hat{V}^{(i)}. \quad (1.9)$$

为了降低生成式 world model 与 value model 的随机性, 论文对每个 action proposal 采样 3 个 future states, 每个 future state 再采样 5 个 value, 因此每个候选动作得到 15 个 value estimates。

聚合不是直接平均, 而是 majority mean: 先用固定阈值判断预测中多数属于 success side 还是 failure side, 然后只对多数一侧的 value 求均值。这样在预测呈双峰或存在少量极端值时, 比 naive mean 更稳定。

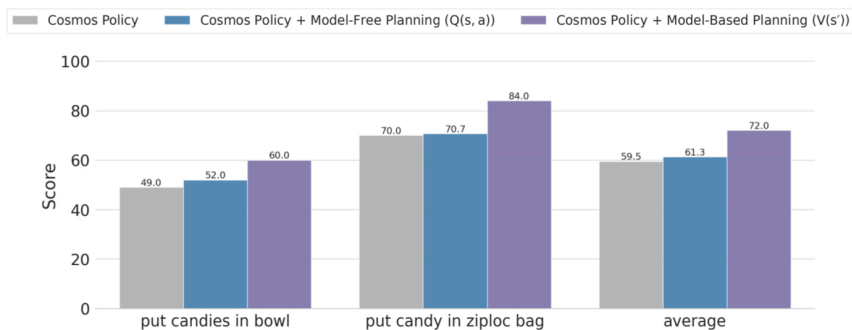


图 1.5: Cosmos Policy 的 model-based planning 结果。利用 world model 预测未来，再用 state value 排序候选动作，能够提升困难真实机器人任务上的完成度。

6.1 $V(s')$ 与 $Q(s, a)$ 的区别

rollout refinement 时，论文可以通过 input masking 让 value prediction 只依赖不同变量：

$$V(s') \text{ 或 } Q(s, a).$$

若使用 $V(s')$ ，必须先经过 world model 得到 future state，再估计 value；这属于 model-based planning。若使用 $Q(s, a)$ ，可以跳过 future-state generation，形成 model-free planning。

论文实验中 $V(s')$ 的 model-based 版本更强，一个重要原因是 world model 把有限 rollout data 中的环境动态作为额外结构利用起来，而不是要求一个高维 $Q(s, a)$ 直接从有限数据中完成全部判断。

7 EDM 噪声分布为什么必须为机器人控制重新设计

Cosmos-Predict2 原始训练近似使用 log-normal 噪声分布：

$$\ln \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2), \quad P_{\text{mean}} = 1.39, \quad P_{\text{std}} = 1.2. \quad (1.10)$$

视频生成允许一定像素或细节误差，但机器人 action 的微小数值偏差可能直接导致抓取失败。更关键的是，采样从

$$\sigma_{\text{max}} = 80$$

的高噪声区域开始；如果训练分布在高 σ 上权重太小，第一步去噪不准确，错误会沿后续步骤继续传播。

因此 Cosmos Policy 使用混合分布：

$$\sigma \sim \begin{cases} \text{原 log-normal}, & \text{概率0.7,} \\ \mathcal{U}(1, 85), & \text{概率0.3.} \end{cases} \quad (1.11)$$

这不是抛弃原视频模型的训练分布，而是在主体保持不变的同时给高噪声端增加更厚的 tail。

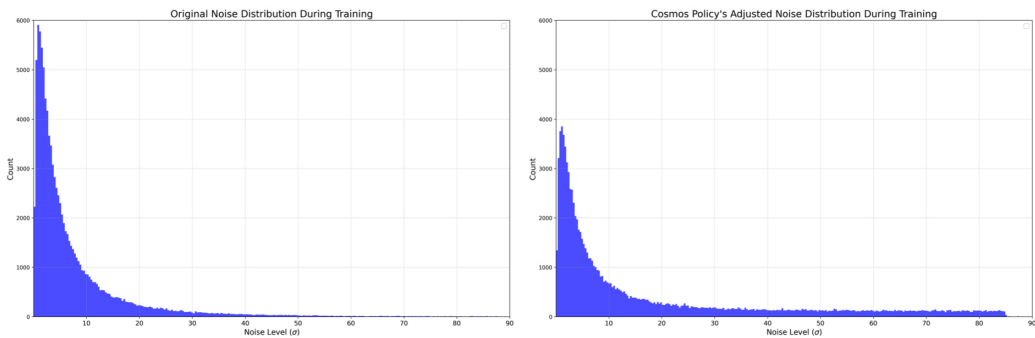


图 1.6: Cosmos-Predict2 原始噪声分布与 Cosmos Policy 的 hybrid log-normal-uniform 分布。后者明显增加高 σ 区域的训练概率。

推理时也做了一个很不直观但重要的修改：把最低噪声提高为

$$\sigma_{\min} = 4,$$

而不是继续去噪到接近 0。作者观察到非常小 σ 下的最后几步反而降低 action、future state 与 value 的数值精度，因此提前停止更好。

8 并行解码与自回归解码

因为训练时既包含 joint prediction，也包含 conditional prediction，同一个模型可以有两种工作方式：

- **Direct policy:** 并行生成 $(a, s', V(s'))$ ，执行时只取 action。LIBERO/RoboCasa 使用 5 denoising steps，ALOHA 使用 10 steps。

- **Planning:** 按 $a \rightarrow s' \rightarrow V(s')$ 自回归生成, 使 future state 和 value 能得到更完整的条件。真实机器人 planning 中, action 使用 10 steps, future state 和 value 分别使用 5 steps。

因此 “policy” 和 “planner” 并不是两个完全不同的网络结构, 而是同一 latent diffusion 模型在 conditioning scheme、checkpoint 和 decoding order 上的不同工作点。

9 几个值得记住的实验结论

这里不抄完整 benchmark 表, 只保留能解释设计的结果:

- 去掉 policy/world model 的 auxiliary targets 后, LIBERO 平均成功率下降约 1.5 个百分点, 说明 future-state/value supervision 对 action policy 本身有帮助。
- 从随机初始化训练而不是从 Cosmos-Predict2 初始化, LIBERO 平均成功率下降约 3.9 个百分点; 在 ALOHA fold-shirt 上也出现明显退化, 说明视频预训练先验确实被迁移到了控制。
- RoboCasa 的逐项消融中, 最后退化到只学 $p(a | s)$ 的 barebones policy 时性能下降最大, 说明 “预测未来世界” 不是装饰性输出。
- 在 RoboCasa 中只用 1 个 denoising step 仍能取得接近多步版本的性能, 同时将单个 action chunk 的延迟降低到约 0.16 s。
- ALOHA 的 best-of- N planning 使用 $N = 8$ 并行候选、每个候选 3 个 future states、每个 future state 5 个 values; 整次搜索约需 4.9 s, 因此目前 planning 更适合较慢的高价值决策, 而不是高频动态控制。

10 把整篇方法串起来

Cosmos Policy 可以压缩成下面八步:

1. 预训练视频模型已经会在 latent sequence 上做 conditional denoising;
2. latent frame injection 把 action、proprioception、value 包装成相同形状的 latent frame;

3. clean/noisy mask 把一个 backbone 切换成 policy、world model 或 value function;
4. policy 不只生成 action, 还联合生成 future state/value, 获得 dynamics-aware auxiliary supervision;
5. Monte Carlo return 为 value latent 提供监督;
6. on-policy rollout 让 world/value 看到专家数据之外的成功和失败区域;
7. best-of- N 用 world model 展开候选未来, 再用 value 排序动作;
8. hybrid noise distribution 与较高 $\sigma_{\min}$ 把视频生成模型的去噪过程重新校准到高精度机器人控制。

11 值得长期保留的理解

- **Latent frame injection** 是最关键的抽象。它把“增加新机器人模态”从修改网络结构的问题, 变成修改 latent sequence 语义的问题。
- **Policy、world model、value function 的统一来自 conditioning, 而不是多个 head。**
- **Future-state prediction 对 policy 本身也有价值。**它强迫动作生成与环境动态保持一致, 而不仅仅拟合 expert action 数值。
- **Demonstration data 与 planning data 的职责不同。**专家数据擅长告诉模型“应该怎么做”, on-policy rollouts 才能告诉 world/value “做错以后会发生什么”。
- **Planning 的核心不是再训练一个 planner network。**Cosmos Policy 直接复用已经统一好的 policy/world/value 生成接口做 best-of- N search。
- **视频 diffusion 的默认 noise schedule 并不天然适合机器人控制。**从视频生成迁移到精确 action generation 后, 噪声分布本身也成为关键策略设计。

资料入口

- 论文: <https://arxiv.org/abs/2601.16163>
- 项目主页: <https://research.nvidia.com/labs/dir/cosmos-policy/>